

1. Introduction

In this project, we are building a machine learning model that predicts whether a customer will churn or not — basically, leave the company or stay. We are using data like age, gender, income, credit score, etc., and training different ML models to see which one performs best. The goal is to help businesses predict customer behavior and reduce loss.

2. Dataset description

We used a dataset with 1,500 rows and 9 columns. The features are:

- Age
- Gender
- Income
- Marital_Status
- Device_Used
- Credit_Score
- Num_Purchases
- Membership_Years
- Churn → Target attribute (1 = churned, 0 = not churned)

There are 9 features in the dataset. There are 13500 data points. Binary classification problem. Because we are predicting two labels from the features. So this is a binary classification problem.

Numerical features: ['Age', 'Income', 'Credit_Score', 'Num_Purchases', 'Membership_Years', 'Churn']

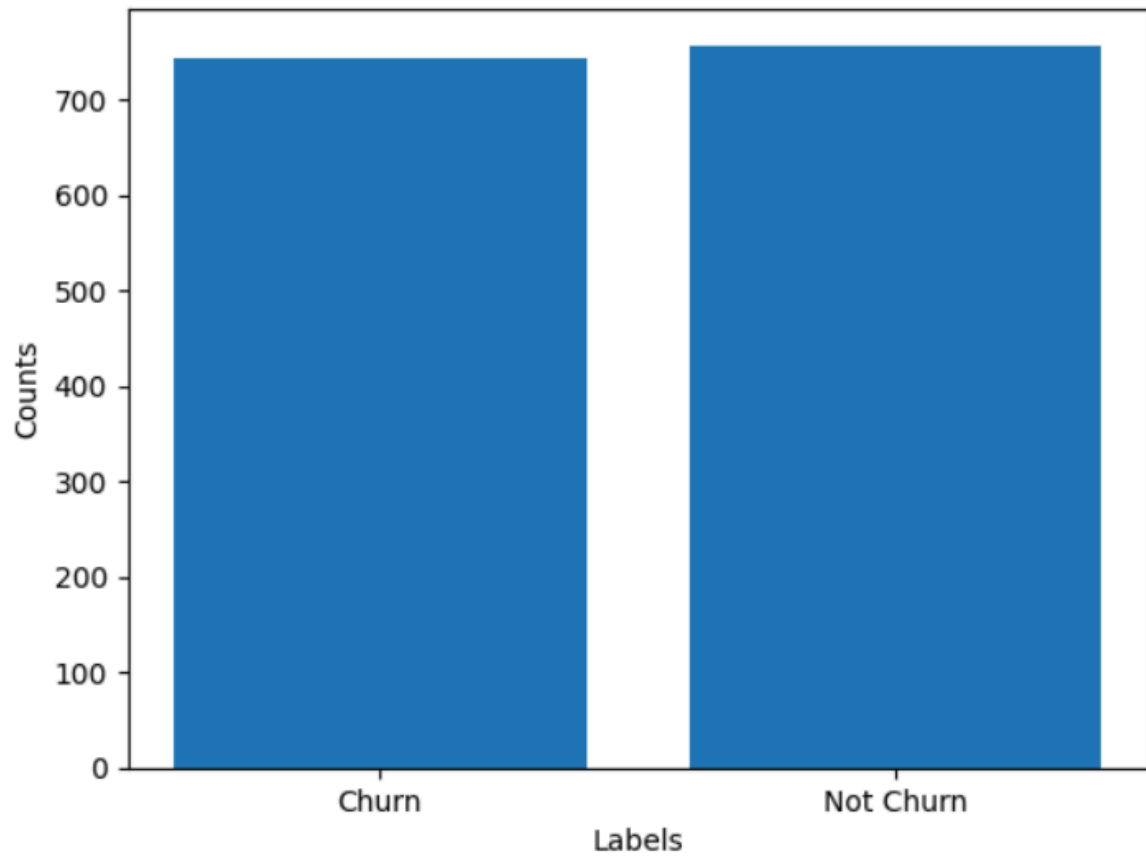
Categorical features: ['Gender', 'Marital_Status', 'Device_Used']

For the output feature, all unique classes do not have an equal number of instances.

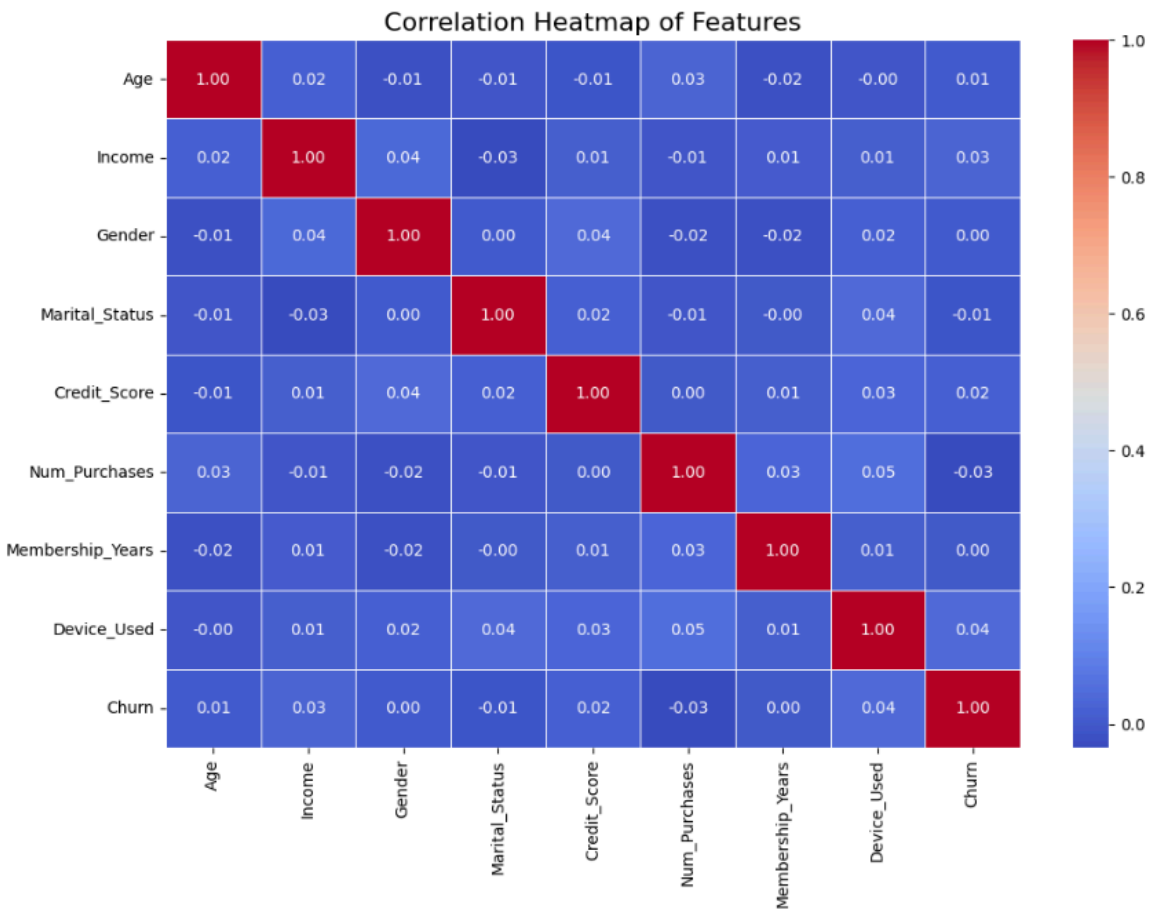
Churn = 0 : 757

Churn = 1: 743

Bar Chart of Labels



Correlation of all the features:



The correlation test helps us understand how strongly features are related to each other and to the target variable (Churn).

From the heatmap, we saw that some features had a positive or negative correlation with Churn. It helps us decide which features are important and which ones might be ignored or removed.

3. Dataset pre-processing

Faults:

- Null / Missing values
Some columns had missing (NaN) values.
- Categorical values
There were features with text-based (object type) data which machine learning models can't handle directly.

- Feature Scaling
The numerical features had different ranges, which can affect model performance, especially for models like Neural Network.

Solutions

- Impute values for missing data
 - If a column was categorical, we filled the missing values with the mode (most frequent value).
 - If a column was numerical, we filled missing values with the median.

This was done to avoid losing data by deleting rows, and because mode/median keeps the distribution safe.

- Encoding categorical variables
 - Used LabelEncoder to convert text-based columns into numeric values.
 - This is necessary because models can't understand strings, and LabelEncoder helps by converting them into 0, 1, 2, etc.
- Feature Scaling
 - Used RobustScaler to scale the numerical features.
 - We did this because some features had outliers, and RobustScaler is good at handling them without being affected much.

4. Dataset splitting

- We split the dataset using stratified sampling, which keeps the balance of the Churn labels in both training and test sets.
- Train set: 70% of the data
- Test set: 30% of the data
- This helps the model learn from a balanced dataset and gives fair results during testing.

5. Model training & testing

We used three models for our classification problem:

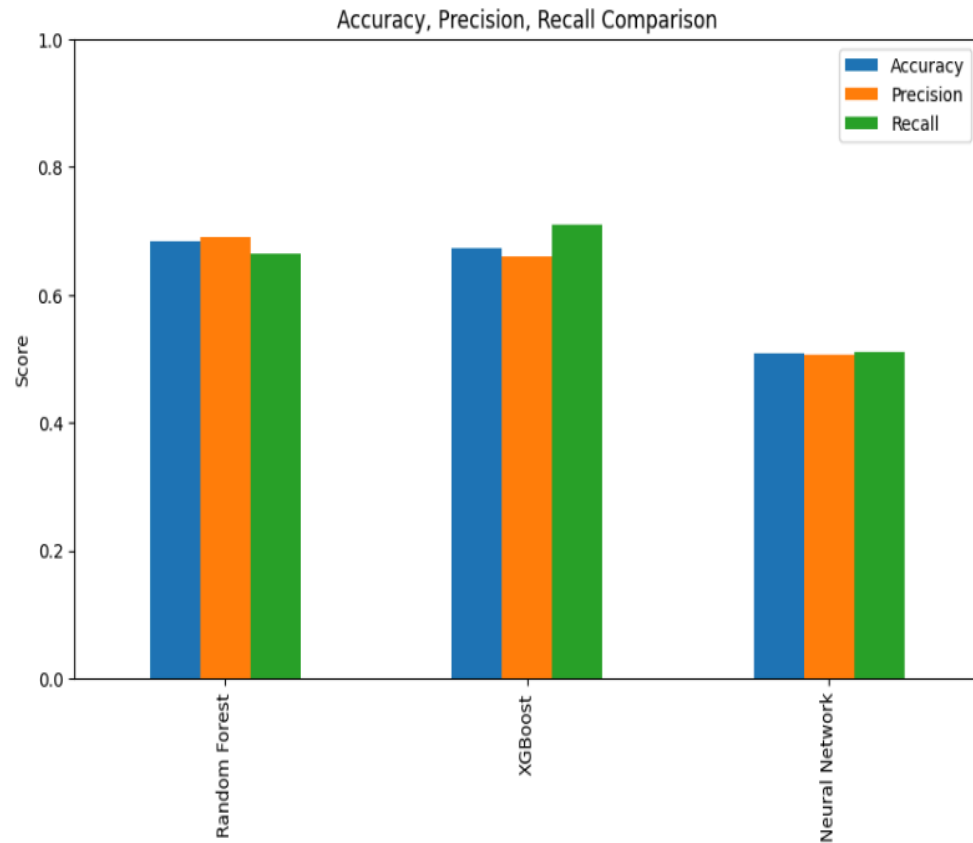
- Neural Network (MLPClassifier)
 - A deep learning model with hidden layers.
 - Trained with early stopping and more iterations to improve learning.
- XGBoost
 - An advanced ensemble learning model based on gradient boosting decision trees.
 - Handles complex patterns, nonlinear relationships, and interactions between features very well.
- Random Forest
 - A powerful ensemble model that builds many decision trees and combines their output.
 - Great for handling complex and nonlinear data.

Each model was trained on the training set and tested on the test set. We used several performance metrics to evaluate them.

6. Model selection/Comparison analysis

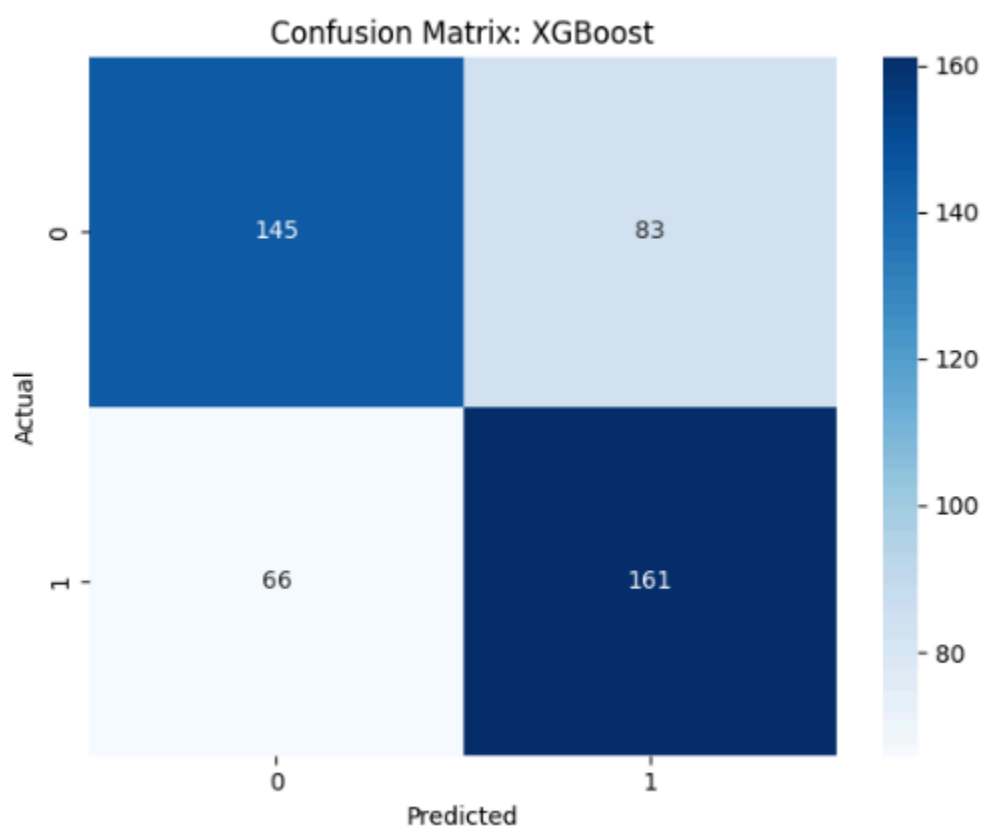
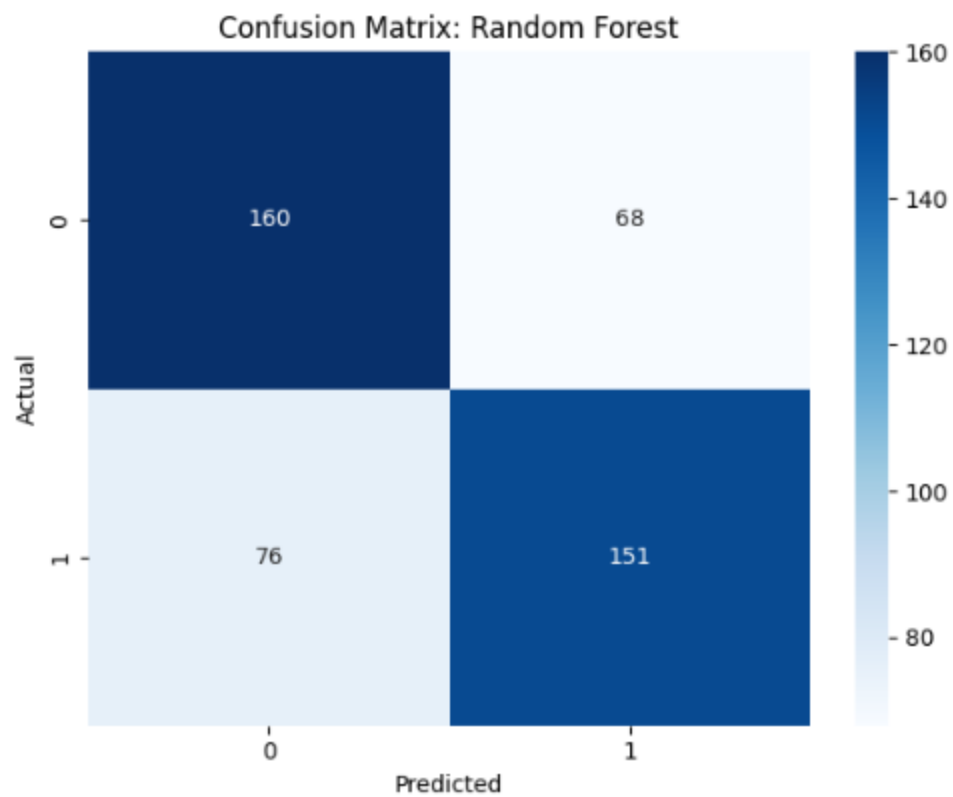
We compared the models using different evaluation methods:

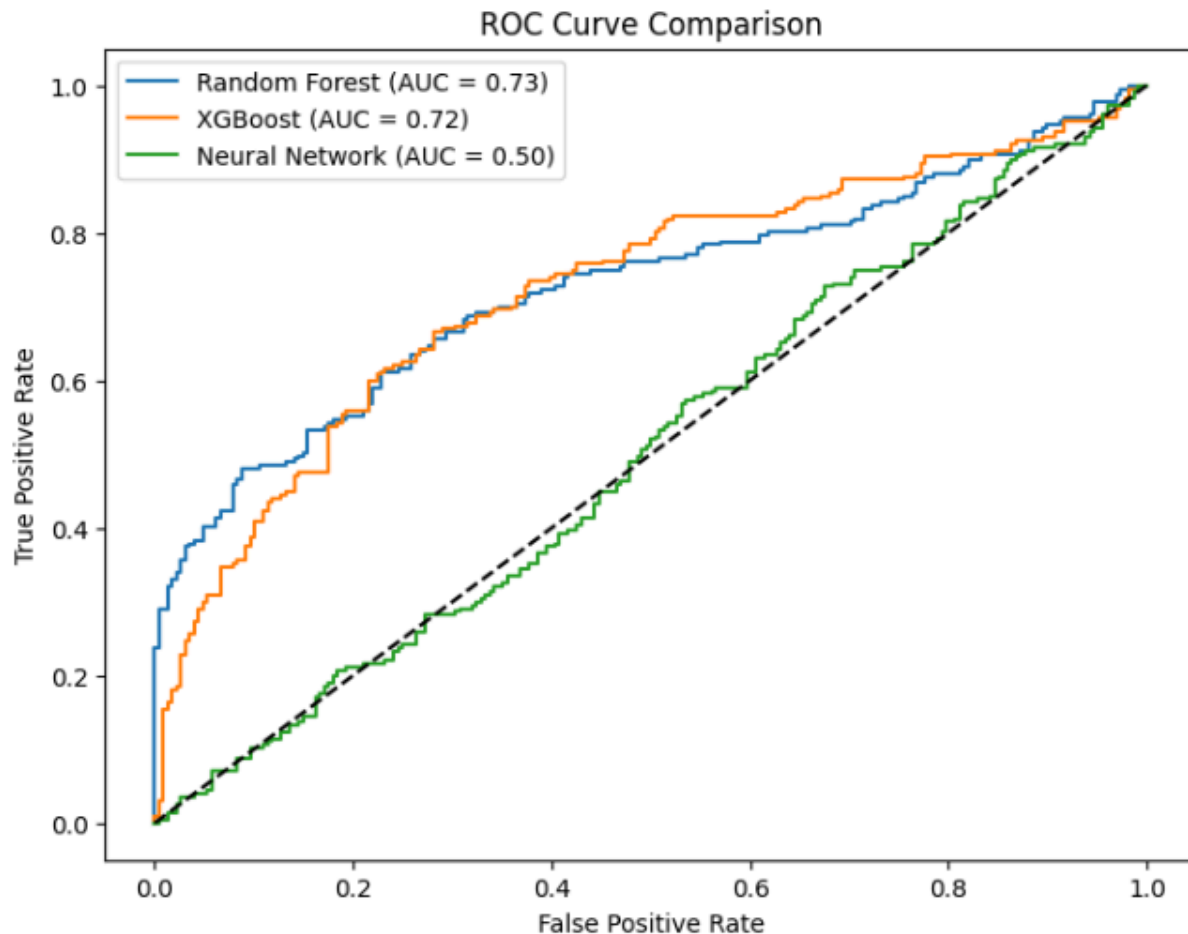
- Bar chart for Accuracy, Precision, Recall
 - We plotted a bar chart to compare accuracy, precision, and recall for each model.
 - This helped to see which model performed better overall.



- **Confusion Matrix:**

- We created confusion matrices for all three models.
- It shows true positives, true negatives, false positives, and false negatives clearly.





7. Conclusion

From the results, we saw that all three models performed quite well after upsampling the dataset to fix the class imbalance.

Random Forest performed the best with the highest accuracy (68.35%) and AUC (72.71%), meaning it predicted churn more effectively than the other models. XGBoost and Neural Network gave results close to random guessing. This shows Random Forest is better suited for this dataset. The main challenges were handling missing values, class imbalance, and model tuning.

We think we got these results because:

- We handled the imbalance in the dataset properly.
- Scaling the features helped models like Neural Network.
- Random Forest worked well because it's good at handling various types of features.

Challenges faced:

- Handling missing values without losing useful data.
- Class imbalance had to be fixed to make fair predictions.
- Evaluating and comparing multiple models took some effort.